Documento de Arquitetura de Software

Vallor Tracker

Versão 10.0 Multi-Tenant Server-Side Production-Ready

Sistema Full-Stack de Rastreamento de Conversões e Atribuição de Marketing

Atualizado em: Janeiro 2026

Sumário Executivo

Vallor Tracker é uma plataforma SaaS de rastreamento de conversões e atribuição de marketing, projetada para agências gerenciarem milhares de clientes de forma isolada e white-label. Concorre diretamente com UTMify, oferecendo rastreamento server-side, integração com Meta CAPI, e atribuição precisa de vendas.

Diferenciais Competitivos

- **Multi-Tenant Nativo:** Isolamento completo de dados por agência e cliente
- **White-Label:** Cada cliente usa seu próprio domínio para rastreamento
- **Server-Side Tracking:** Envio direto para Meta Conversions API (CAPI)
- **Advanced Matching:** Captura e hash SHA256 de email, telefone, nome, cidade
- **Atribuição Precisa:** Match por fbc, fbp, fingerprint, IP com janela de 30 dias
- **Dashboard Power BI:** Visualização de ROI, funil de vendas e performance por criativo
- **Integrações Nativas:** Kiwify, Hotmart, Eduzz via webhooks

Índice

1. Visão Geral da Arquitetura
2. Arquitetura Multi-Tenant e White-Label
3. Stack Tecnológica
4. Modelagem de Dados Completa
5. Motor de Rastreamento (Engine 2026)
6. API Routes (Next.js)
7. Inteligência de Atribuição
8. Dashboard Analítico
9. Segurança e LGPD
10. Escalabilidade e Performance
11. Deploy e Operação
12. Código Consolidado

1. Visão Geral da Arquitetura

1.1 Visão do Sistema

O Vallor Tracker é um sistema de rastreamento e atribuição de marketing que opera em três camadas principais:

1. **Camada de Captura (Frontend):** Script JavaScript instalado nas landing pages dos clientes

2. **Camada de Processamento (Backend):** API Routes Next.js para ingestão e processamento de eventos
3. **Camada de Apresentação (Dashboard):** Interface React com Tremor para visualização de métricas

Fluxo de Dados End-to-End

```
┌──────────────────┐
│  Landing Page    │  ← Visitante clica no anúncio do Facebook
│  (Cliente)       │
└────────┬─────────┘
         │ 1. Script vallor-tracker.js captura:
         │    - fbp, fbc, IP, User Agent
         │    - UTM parameters
         │    - Fingerprint do navegador
         │
         ▼
┌──────────────────┐
│  API /api/track  │  ← 2. Envia evento PageView
│  (Next.js)       │
└────────┬─────────┘
         │ 3. Resolve tenant por script_key
         │ 4. Insere em tracking_events
         │ 5. Envia para Meta CAPI (async)
         │
         ▼
┌──────────────────┐
│  Supabase        │
│  (PostgreSQL)    │
└────────┬─────────┘
         │
         │ [Usuário compra no checkout]
         │
         ▼
┌──────────────────┐
│  Kiwify/Hotmart  │  ← 6. Plataforma de checkout
│  (Webhook)       │
└────────┬─────────┘
         │ 7. POST para /api/webhook/kiwify
         │
         ▼
┌──────────────────┐
│  Atribuição      │  ← 8. Match por fbp, IP, email
│  (Logic Layer)   │     9. Cria registro em conversions
└────────┬─────────┘     10. Atribui à campanha/anúncio
         │               11. Envia Purchase para Meta CAPI
         │
         ▼
┌──────────────────┐
│  Dashboard       │  ← 12. Visualiza ROAS, CPA, funil
│  (React/Tremor)  │
└──────────────────┘
```

1.2 Componentes Principais

Componente	Tecnologia	Responsabilidade
Script de Rastreamento	JavaScript Vanilla	Captura eventos client-side e envia para API
API de Ingestão	Next.js Route Handlers	Recebe, valida e persiste eventos
Motor de Atribuição	TypeScript	Match de conversões com eventos originais
Meta CAPI Client	TypeScript/Fetch API	Envia eventos para Meta Conversions API
Webhooks Receiver	Next.js Route Handlers	Processa notificações de checkouts
Dashboard	React + Tremor	Visualização de métricas e KPIs
Banco de Dados	PostgreSQL (Supabase)	Armazenamento de eventos, conversões e configurações

2. Arquitetura Multi-Tenant e White-Label

2.1 Conceitos Fundamentais

Agência (Conta Master)

A agência é o tenant principal do sistema. Cada agência:

- Cria e gerencia múltiplos clientes (subcontas)
- Controla usuários internos e suas permissões
- Configura integrações globais
- Acessa dados consolidados ou por cliente

Cliente (Subconta)

Cada cliente pertence a uma agência e possui:

- **Identificação única:** UUID isolado por agência
- **Domínios white-label:** tracker.cliente-a.com, analytics.cliente-b.com
- **Script personalizado:** Chave única (script_key) para rastreamento
- **Configurações próprias:** Meta Pixel ID, Access Token, fuso horário
- **Isolamento de dados:** Eventos e conversões isolados por client_id

2.2 Resolução de Tenant

O sistema resolve qual cliente está enviando dados através de:

Método	Prioridade	Caso de Uso
`script_key`	1 (Mais alta)	Enviado no payload do evento pelo script
Domínio (Referer/Origin)	2	Quando o script está em domínio white-label
Token de API (Bearer)	3	Webhooks e integrações server-to-server

2.3 Isolamento de Dados

O isolamento é garantido por múltiplas camadas:

A. Colunas de Tenant

Todas as tabelas contêm:

```
agency_id UUID NOT NULL REFERENCES agencies(id)
client_id UUID NOT NULL REFERENCES clients(id)
```

B. Row Level Security (RLS)

Políticas PostgreSQL impedem acesso não autorizado:

```
-- Exemplo de política RLS
CREATE POLICY "clients_select"
ON public.clients
FOR SELECT
USING (public.has_client_access(id));

-- Função de autorização
CREATE FUNCTION has_client_access(check_client_id UUID)
RETURNS BOOLEAN AS $$
  SELECT EXISTS (
    SELECT 1
    FROM clients c
    JOIN tenant_users tu ON tu.agency_id = c.agency_id
    WHERE c.id = check_client_id
      AND tu.user_id = auth.uid()
      AND (tu.client_id IS NULL OR tu.client_id = check_client_id)
  );
$$ LANGUAGE sql SECURITY DEFINER;
```

C. Validação em Camada de Aplicação

Todas as queries incluem filtro por tenant:

```
// Exemplo em TypeScript
const { data: events } = await supabase
  .from('tracking_events')
  .select('*')
  .eq('client_id', currentClientId) // Sempre filtrar por tenant
  .order('event_time', { ascending: false });
```

3. Stack Tecnológica

Frontend

- **Framework:** Next.js 14+ (App Router)
- **Linguagem:** TypeScript 5+
- **Estilização:** Tailwind CSS 3
- **UI Components:** Shadcn/UI
- **Gráficos:** Tremor (Charts) + Recharts
- **Ícones:** Lucide React

Backend

- **Runtime:** Next.js API Routes (Edge/Node)
- **Linguagem:** TypeScript 5+
- **Validação:** Zod
- **HTTP Client:** Fetch API (nativo)
- **Criptografia:** Crypto API (SHA256)
- **Rate Limiting:** Upstash Redis

Banco de Dados

- **SGBD:** PostgreSQL 15+ (via Supabase)
- **ORM:** Supabase Client (@supabase/ssr)
- **Auth:** Supabase Auth (JWT)
- **Segurança:** Row Level Security (RLS)
- **Realtime:** Supabase Realtime (opcional)

Infraestrutura

- **Hospedagem:** Vercel (Edge Network)
- **CI/CD:** GitHub Actions + Vercel
- **Cache:** Upstash Redis
- **Filas:** Upstash QStash
- **Monitoramento:** Vercel Analytics + Sentry
- **DNS:** Cloudflare (White-label)

3.1 Integrações Externas

Serviço	Finalidade	Protocolo
Meta Conversions API	Envio de eventos server-side	HTTPS REST
Kiwify	Notificações de compra via webhook	HTTPS Webhook
Hotmart	Notificações de compra via webhook	HTTPS Webhook
Eduzz	Notificações de compra via webhook	HTTPS Webhook
IPify	Captura de IP público do visitante	HTTPS REST

4. Modelagem de Dados Completa

4.1 Diagrama de Entidades

```
┌─────────────────┐
│    agencies     │
│─────────────────│
│ id (PK)         │
│ name            │
│ slug (unique)   │
│ logo_url        │
└────────┬────────┘
         │ 1
         │
         │ N
         ▼
┌─────────────────┐         ┌──────────────────┐
│    clients      │────N────│  client_domains  │
│─────────────────│         │──────────────────│
│ id (PK)         │         │ id (PK)          │
│ agency_id (FK)  │         │ client_id (FK)   │
│ name            │         │ domain (unique)  │
│ meta_pixel_id   │         │ script_key       │
│ meta_token      │         └──────────────────┘
└────────┬────────┘
         │ 1
         │
         │ N
         ▼
┌─────────────────┐         ┌──────────────────┐
│    visitors     │────1────│ tracking_events  │
│─────────────────│    N    │──────────────────│
│ id (PK)         │         │ id (PK)          │
│ client_id (FK)  │         │ client_id (FK)   │
│ fbp             │         │ visitor_id (FK)  │
│ fbc             │         │ event_name       │
│ ip              │         │ event_id (unique)│
│ fingerprint     │         │ event_time       │
│ email_hash      │         │ properties       │
└────────┬────────┘         └──────────────────┘
         │ 1
         │
         │ N
         ▼
┌─────────────────┐
│   conversions   │
│─────────────────│
│ id (PK)         │
│ client_id (FK)  │
│ visitor_id (FK) │
│ provider        │
│ order_id        │
│ amount          │
│ attributed_by   │
└─────────────────┘
```

4.2 SQL Completo do Schema

A. Extensões e Funções Auxiliares

```
-- Extensões necessárias
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;

-- Função para atualizar updated_at automaticamente
CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  NEW.updated_at = timezone('utc', now());
  RETURN NEW;
END;
$$;
```

B. Tabelas de Tenant

```
-- Agências (Conta Master)
CREATE TABLE IF NOT EXISTS public.agencies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug CITEXT NOT NULL UNIQUE,
  logo_url TEXT,
  white_label_name TEXT,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

-- Clientes (Subcontas)
CREATE TABLE IF NOT EXISTS public.clients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  slug CITEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
  timezone TEXT NOT NULL DEFAULT 'America/Sao_Paulo',
  currency TEXT NOT NULL DEFAULT 'BRL',

  -- Configurações Meta
  meta_pixel_id TEXT,
  meta_access_token TEXT,
  meta_test_event_code TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT clients_agency_slug_unique UNIQUE (agency_id, slug)
);

-- Domínios White-Label
CREATE TABLE IF NOT EXISTS public.client_domains (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  domain CITEXT NOT NULL UNIQUE,
  is_primary BOOLEAN NOT NULL DEFAULT false,
  script_key TEXT NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);
```

C. Tabelas de Campanha e Mídia

```
-- Campanhas
CREATE TABLE IF NOT EXISTS public.campaigns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  external_id TEXT, -- ID da campanha no Facebook
  platform TEXT NOT NULL DEFAULT 'meta',
  source TEXT, -- utm_source
  medium TEXT, -- utm_medium
  name TEXT NOT NULL,
  spend NUMERIC(14,2) NOT NULL DEFAULT 0,
  starts_at TIMESTAMPTZ,
  ends_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT campaigns_unique_external UNIQUE (client_id, platform, external_id)
);

-- Conjuntos de Anúncios
CREATE TABLE IF NOT EXISTS public.adsets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  campaign_id UUID NOT NULL REFERENCES public.campaigns(id) ON DELETE CASCADE,
  external_id TEXT,
  name TEXT NOT NULL,
  spend NUMERIC(14,2) NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT adsets_unique_external UNIQUE (client_id, campaign_id, external_id)
);

-- Anúncios
CREATE TABLE IF NOT EXISTS public.ads (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  adset_id UUID NOT NULL REFERENCES public.adsets(id) ON DELETE CASCADE,
  campaign_id UUID NOT NULL REFERENCES public.campaigns(id) ON DELETE CASCADE,
  external_id TEXT,
  name TEXT NOT NULL,
  spend NUMERIC(14,2) NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT ads_unique_external UNIQUE (client_id, adset_id, external_id)
);
```

D. Tabelas de Rastreamento (CORE)

```
-- Visitantes (Identidade Agregada)
CREATE TABLE IF NOT EXISTS public.visitors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,

  -- Identificadores de Marketing
  external_id TEXT, -- ID do CRM/Lead
  click_id TEXT, -- fbclid ou similar

  -- Dados Técnicos
  ip INET,
  user_agent TEXT,
  fingerprint TEXT, -- Raw fingerprint
  fingerprint_hash TEXT, -- Hash do fingerprint

  -- Facebook Browser/Click ID
  fbp TEXT,
  fbc TEXT,

  -- Advanced Matching (em Hash SHA256)
  email_hash TEXT,
  phone_hash TEXT,
  first_name_hash TEXT,
  last_name_hash TEXT,

  -- Localização
  city TEXT,
  state TEXT,
  zip TEXT,
  country TEXT DEFAULT 'br',

  -- Primeira Interação
  landing_url TEXT,
  referrer TEXT,
  utm_source TEXT,
  utm_medium TEXT,
  utm_campaign TEXT,
  utm_content TEXT,
  utm_term TEXT,

  -- Timestamps
  first_seen_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  last_seen_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

-- Eventos de Tracking
CREATE TABLE IF NOT EXISTS public.tracking_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  visitor_id UUID REFERENCES public.visitors(id) ON DELETE SET NULL,
  campaign_id UUID REFERENCES public.campaigns(id) ON DELETE SET NULL,
  adset_id UUID REFERENCES public.adsets(id) ON DELETE SET NULL,
  ad_id UUID REFERENCES public.ads(id) ON DELETE SET NULL,

  -- Evento
  event_name TEXT NOT NULL CHECK (event_name IN (
    'PageView', 'Checkout', 'Lead', 'Purchase',
    'AddToCart', 'InitiateCheckout', 'CompleteRegistration', 'ViewContent'
  )),
  event_time TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  event_id TEXT NOT NULL, -- Para deduplicação com Pixel

  -- Contexto
  source_url TEXT, -- URL que enviou o script
```

```
  page_url TEXT, -- URL da página
  referrer TEXT,

  -- Dados Capturados
  ip INET,
  user_agent TEXT,
  fingerprint_hash TEXT,
  external_id TEXT,
  click_id TEXT,
  fbp TEXT,
  fbc TEXT,

  -- UTM
  utm_source TEXT,
  utm_medium TEXT,
  utm_campaign TEXT,
  utm_content TEXT,
  utm_term TEXT,

  -- Dados Customizados
  properties JSONB NOT NULL DEFAULT '{}'::JSONB,

  -- Status
  meta_sent BOOLEAN NOT NULL DEFAULT false,

  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT tracking_events_event_unique UNIQUE (client_id, event_id)
);

-- Conversões (Compras)
CREATE TABLE IF NOT EXISTS public.conversions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID NOT NULL REFERENCES public.clients(id) ON DELETE CASCADE,
  visitor_id UUID REFERENCES public.visitors(id) ON DELETE SET NULL,
  tracking_event_id UUID REFERENCES public.tracking_events(id) ON DELETE SET NULL,
  campaign_id UUID REFERENCES public.campaigns(id) ON DELETE SET NULL,
  adset_id UUID REFERENCES public.adsets(id) ON DELETE SET NULL,
  ad_id UUID REFERENCES public.ads(id) ON DELETE SET NULL,

  -- Origem da Conversão
  provider TEXT NOT NULL, -- kiwify, hotmart, eduzz, generic
  order_id TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'approved' CHECK (status IN (
    'approved', 'paid', 'completed', 'pending', 'refunded', 'cancelled', 'chargeback'
  )),

  -- Valor
  currency TEXT NOT NULL DEFAULT 'BRL',
  amount NUMERIC(14,2) NOT NULL DEFAULT 0,

  -- Evento
  event_name TEXT NOT NULL DEFAULT 'Purchase',
  event_id TEXT NOT NULL,

  -- Dados para Match
  external_id TEXT,
  click_id TEXT,
  fbp TEXT,
  fbc TEXT,
  ip INET,
  user_agent TEXT,
  fingerprint_hash TEXT,

  -- Dados do Cliente (para Advanced Matching)
  customer_email TEXT,
  customer_phone TEXT,
  customer_first_name TEXT,
  customer_last_name TEXT,
  customer_city TEXT,
  customer_state TEXT,
  customer_zip TEXT,
  customer_country TEXT,

  -- Payload Original
  raw_payload JSONB NOT NULL DEFAULT '{}'::JSONB,

  -- Atribuição
  attributed_by TEXT, -- fbc, fbp, ip, fingerprint, email, unattributed
  attributed_at TIMESTAMPTZ,

  -- Timestamps
  conversion_time TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  meta_sent BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
```

```
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),

  CONSTRAINT conversions_unique_order UNIQUE (client_id, provider, order_id),
  CONSTRAINT conversions_unique_event UNIQUE (client_id, event_id)
);
```

E. Tabelas de Integração

```
-- Logs de Webhook
CREATE TABLE IF NOT EXISTS public.webhook_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID REFERENCES public.agencies(id) ON DELETE SET NULL,
  client_id UUID REFERENCES public.clients(id) ON DELETE SET NULL,
  provider TEXT NOT NULL,
  event_type TEXT,
  external_event_id TEXT,
  status TEXT NOT NULL DEFAULT 'received' CHECK (status IN (
    'received', 'processing', 'processed', 'duplicate', 'rejected', 'error'
  )),
  request_headers JSONB NOT NULL DEFAULT '{}'::JSONB,
  payload JSONB NOT NULL DEFAULT '{}'::JSONB,
  response_body JSONB,
  error_message TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

-- Tokens de API
CREATE TABLE IF NOT EXISTS public.api_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID REFERENCES public.clients(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  token_hash TEXT NOT NULL UNIQUE,
  scopes TEXT[] NOT NULL DEFAULT ARRAY['webhook:purchase']::TEXT[],
  last_used_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

-- Usuários do Tenant
CREATE TABLE IF NOT EXISTS public.tenant_users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agency_id UUID NOT NULL REFERENCES public.agencies(id) ON DELETE CASCADE,
  client_id UUID REFERENCES public.clients(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  role TEXT NOT NULL DEFAULT 'viewer' CHECK (role IN ('owner', 'admin', 'analyst', 'viewer')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now()),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

CREATE UNIQUE INDEX IF NOT EXISTS tenant_users_agency_unique
ON public.tenant_users(user_id, agency_id)
WHERE client_id IS NULL;

CREATE UNIQUE INDEX IF NOT EXISTS tenant_users_client_unique
ON public.tenant_users(user_id, client_id)
WHERE client_id IS NOT NULL;
```

F. Índices de Performance

```
-- Índices para Clientes e Domínios
CREATE INDEX IF NOT EXISTS idx_clients_agency_id ON public.clients(agency_id);
CREATE INDEX IF NOT EXISTS idx_client_domains_client_id ON public.client_domains(client_id);
CREATE INDEX IF NOT EXISTS idx_client_domains_script_key ON public.client_domains(script_key);

-- Índices para Campanhas
CREATE INDEX IF NOT EXISTS idx_campaigns_client_id ON public.campaigns(client_id);
CREATE INDEX IF NOT EXISTS idx_adsets_client_id ON public.adsets(client_id);
CREATE INDEX IF NOT EXISTS idx_ads_client_id ON public.ads(client_id);

-- Índices para Visitantes (CRÍTICOS para Atribuição)
CREATE INDEX IF NOT EXISTS idx_visitors_client_last_seen
ON public.visitors(client_id, last_seen_at DESC);

CREATE INDEX IF NOT EXISTS idx_visitors_client_external_id
ON public.visitors(client_id, external_id) WHERE external_id IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_visitors_client_fbp
ON public.visitors(client_id, fbp) WHERE fbp IS NOT NULL;
```

```
CREATE INDEX IF NOT EXISTS idx_visitors_client_fbc
ON public.visitors(client_id, fbc) WHERE fbc IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_visitors_client_fingerprint
ON public.visitors(client_id, fingerprint_hash) WHERE fingerprint_hash IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_visitors_client_ip_ua
ON public.visitors(client_id, ip, user_agent);

CREATE INDEX IF NOT EXISTS idx_visitors_client_email_hash
ON public.visitors(client_id, email_hash) WHERE email_hash IS NOT NULL;

-- Índices para Eventos
CREATE INDEX IF NOT EXISTS idx_tracking_events_client_time
ON public.tracking_events(client_id, event_time DESC);

CREATE INDEX IF NOT EXISTS idx_tracking_events_client_name_time
ON public.tracking_events(client_id, event_name, event_time DESC);

CREATE INDEX IF NOT EXISTS idx_tracking_events_visitor_time
ON public.tracking_events(visitor_id, event_time DESC);

CREATE INDEX IF NOT EXISTS idx_tracking_events_client_fbp
ON public.tracking_events(client_id, fbp) WHERE fbp IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_tracking_events_client_fbc
ON public.tracking_events(client_id, fbc) WHERE fbc IS NOT NULL;

-- Índices para Conversões
CREATE INDEX IF NOT EXISTS idx_conversions_client_time
ON public.conversions(client_id, conversion_time DESC);

CREATE INDEX IF NOT EXISTS idx_conversions_client_status_time
ON public.conversions(client_id, status, conversion_time DESC);

CREATE INDEX IF NOT EXISTS idx_conversions_client_campaign_time
ON public.conversions(client_id, campaign_id, conversion_time DESC);

-- Índices para Logs
CREATE INDEX IF NOT EXISTS idx_webhook_logs_client_created
ON public.webhook_logs(client_id, created_at DESC);
```

G. Triggers e Funções RLS

```
-- Triggers para updated_at
CREATE TRIGGER set_agencies_updated_at
BEFORE UPDATE ON public.agencies
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

CREATE TRIGGER set_clients_updated_at
BEFORE UPDATE ON public.clients
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

-- ... (criar para todas as tabelas)

-- Funções de Autorização
CREATE OR REPLACE FUNCTION public.is_agency_member(check_agency_id UUID)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.tenant_users tu
    WHERE tu.user_id = auth.uid()
      AND tu.agency_id = check_agency_id
  );
$$;

CREATE OR REPLACE FUNCTION public.has_client_access(check_client_id UUID)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.clients c
    JOIN public.tenant_users tu ON tu.agency_id = c.agency_id
    WHERE c.id = check_client_id
```

```
      AND tu.user_id = auth.uid()
      AND (tu.client_id IS NULL OR tu.client_id = check_client_id)
  );
$$;

-- Habilitar RLS
ALTER TABLE public.agencies ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.clients ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tracking_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.conversions ENABLE ROW LEVEL SECURITY;

-- Políticas de Acesso
CREATE POLICY "agencies_select"
ON public.agencies
FOR SELECT
USING (public.is_agency_member(id));

CREATE POLICY "clients_select"
ON public.clients
FOR SELECT
USING (public.has_client_access(id));

CREATE POLICY "tracking_events_access"
ON public.tracking_events
FOR ALL
USING (public.has_client_access(client_id))
WITH CHECK (public.has_client_access(client_id));

CREATE POLICY "conversions_access"
ON public.conversions
FOR ALL
USING (public.has_client_access(client_id))
WITH CHECK (public.has_client_access(client_id));
```

5. Motor de Rastreamento (Engine 2026)

> ⚠ **Importante:** Este script deve ser minificado e servido via CDN white-label para cada cliente.

5.1 Arquivo: `public/tracker/vallor-tracker.js`

```
/**
 * Vallor Tracker - Engine de Rastreamento Avançado 2026
 * @version 10.0.0
 * @license MIT
 */
(function() {
  'use strict';

  const VallorTracker = {
    // Configuração
    config: {
      apiUrl: null,
      scriptKey: null,
      cookieDuration: 30, // dias
      debug: false
    },

    // Utilitários de Cookie
    getCookie: function(name) {
      const match = document.cookie.match('(^|;) ?' + name + '=([^;]*)(;|$)');
      return match ? decodeURIComponent(match[2]) : null;
    },

    setCookie: function(name, value, days) {
      const expires = new Date();
      expires.setTime(expires.getTime() + (days * 24 * 60 * 60 * 1000));
      const cookieValue = encodeURIComponent(value);
      document.cookie = `${name}=${cookieValue};expires=${expires.toUTCString()};path=/;SameSite=Lax`;
    },

    // Gerar Event ID único (para deduplicação)
    generateEventID: function(prefix = 'ev') {
      return `${prefix}_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
    },

    // Criptografia SHA256
    sha256: async function(message) {
      const msgBuffer = new TextEncoder().encode(message);
```

```
    const hashBuffer = await crypto.subtle.digest('SHA-256', msgBuffer);
    const hashArray = Array.from(new Uint8Array(hashBuffer));
    return hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
  },

  // Capturar Fingerprint do Navegador
  getFingerprint: function() {
    const components = [
      navigator.userAgent,
      navigator.language,
      screen.width + 'x' + screen.height,
      screen.colorDepth,
      new Date().getTimezoneOffset(),
      !!window.sessionStorage,
      !!window.localStorage,
      navigator.hardwareConcurrency || 'unknown',
      navigator.platform
    ];
    return btoa(components.join('|'));
  },

  // Capturar Parâmetros UTM
  getUTMParams: function() {
    const params = new URLSearchParams(window.location.search);
    return {
      utm_source: params.get('utm_source'),
      utm_medium: params.get('utm_medium'),
      utm_campaign: params.get('utm_campaign'),
      utm_content: params.get('utm_content'),
      utm_term: params.get('utm_term'),
      fbclid: params.get('fbclid'),
      gclid: params.get('gclid')
    };
  },

  // Capturar IP (via serviço externo ou servidor)
  captureIP: async function() {
    try {
      const response = await fetch('https://api.ipify.org?format=json', {
        timeout: 3000
      });
      const data = await response.json();
      return data.ip;
    } catch (e) {
      if (this.config.debug) console.warn('Vallor Tracker: Could not capture IP', e);
      return '0.0.0.0';
    }
  },

  // Capturar todos os dados do navegador
  captureBrowserData: async function() {
    const utmParams = this.getUTMParams();
    const ip = await this.captureIP();
    const fingerprint = this.getFingerprint();
    const fingerprintHash = await this.sha256(fingerprint);

    // Gerar ou recuperar _fbp (Facebook Browser ID)
    let fbp = this.getCookie('_fbp');
    if (!fbp) {
      fbp = `fb.1.${Date.now()}.${Math.random().toString().substring(2, 11)}`;
      this.setCookie('_fbp', fbp, 90);
    }

    // Capturar _fbc (Facebook Click ID) se existe fbclid
    let fbc = this.getCookie('_fbc');
    if (utmParams.fbclid && !fbc) {
      fbc = `fb.1.${Date.now()}.${utmParams.fbclid}`;
      this.setCookie('_fbc', fbc, 90);
    }

    return {
      fbc: fbc,
      fbp: fbp,
      user_agent: navigator.userAgent,
      ip: ip,
      url: window.location.href,
      referrer: document.referrer,
      fingerprint: fingerprint,
      fingerprint_hash: fingerprintHash,
      screen_resolution: `${screen.width}x${screen.height}`,
      language: navigator.language,
      timezone: Intl.DateTimeFormat().resolvedOptions().timeZone,
      ...utmParams
    };
  },

  // Persistir dados em LocalStorage (janela de 30 dias)
```

```
persistData: function(data) {
  const storageKey = 'vallor_tracker_data';
  const existing = JSON.parse(localStorage.getItem(storageKey) || '{}');
  const merged = { ...existing, ...data, last_updated: Date.now() };
  localStorage.setItem(storageKey, JSON.stringify(merged));
  return merged;
},

// Recuperar dados persistidos
getPersistedData: function() {
  const storageKey = 'vallor_tracker_data';
  const data = localStorage.getItem(storageKey);
  if (!data) return null;

  const parsed = JSON.parse(data);
  const age = Date.now() - (parsed.last_updated || 0);
  const maxAge = 30 * 24 * 60 * 60 * 1000; // 30 dias

  if (age > maxAge) {
    localStorage.removeItem(storageKey);
    return null;
  }

  return parsed;
},

// Capturar dados de formulário (Advanced Matching)
captureFormData: async function(form) {
  const data = {};

  // Buscar campos comuns
  const emailField = form.querySelector('input[type="email"], input[name*="email"], input[id*="email"]');
  const phoneField = form.querySelector('input[type="tel"], input[name*="phone"], input[name*="telefone"]');
  const nameField = form.querySelector('input[name*="name"], input[name*="nome"]');
  const firstNameField = form.querySelector('input[name*="first"], input[name*="primeiro"]');
  const lastNameField = form.querySelector('input[name*="last"], input[name*="ultimo"], input[name*="sobrenome"]');
  const cityField = form.querySelector('input[name*="city"], input[name*="cidade"]');
  const stateField = form.querySelector('input[name*="state"], input[name*="estado"], select[name*="uf"]');
  const zipField = form.querySelector('input[name*="zip"], input[name*="cep"], input[name*="postal"]');

  // Hash dos dados sensíveis
  if (emailField && emailField.value) {
    data.email_hash = await this.sha256(emailField.value.toLowerCase().trim());
  }

  if (phoneField && phoneField.value) {
    const phone = phoneField.value.replace(/\D/g, ''); // Remove formatação
    data.phone_hash = await this.sha256(phone);
  }

  if (firstNameField && firstNameField.value) {
    data.first_name_hash = await this.sha256(firstNameField.value.toLowerCase().trim());
  } else if (nameField && nameField.value) {
    const nameParts = nameField.value.trim().split(' ');
    data.first_name_hash = await this.sha256(nameParts[0].toLowerCase());
    if (nameParts.length > 1) {
      data.last_name_hash = await this.sha256(nameParts[nameParts.length - 1].toLowerCase());
    }
  }

  if (lastNameField && lastNameField.value) {
    data.last_name_hash = await this.sha256(lastNameField.value.toLowerCase().trim());
  }

  // Localização (não precisa de hash)
  if (cityField && cityField.value) {
    data.city = cityField.value.toLowerCase().trim();
  }

  if (stateField && stateField.value) {
    data.state = stateField.value.toLowerCase().trim();
  }

  if (zipField && zipField.value) {
    data.zip = zipField.value.replace(/\D/g, '');
  }

  data.country = 'br';

  return data;
},

// Enviar evento para API
sendEvent: async function(eventName, customData = {}) {
  if (!this.config.apiUrl || !this.config.scriptKey) {
    console.error('Vallor Tracker: Configuration missing');
    return null;
```

```
    }

    const browserData = await this.captureBrowserData();
    const eventId = this.generateEventID();

    // Persistir dados importantes
    this.persistData({
      fbp: browserData.fbp,
      fbc: browserData.fbc,
      fingerprint_hash: browserData.fingerprint_hash,
      last_event: eventName,
      last_event_time: Date.now()
    });

    const payload = {
      script_key: this.config.scriptKey,
      event_name: eventName,
      event_time: Math.floor(Date.now() / 1000),
      event_id: eventId,
      ...browserData,
      ...customData
    };

    // 1. Disparar via Meta Pixel (se existir) - Browser Side
    if (typeof fbq !== 'undefined') {
      fbq('track', eventName, customData, { eventID: eventId });
    }

    // 2. Enviar para nosso servidor - Server Side
    try {
      const response = await fetch(this.config.apiUrl, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload),
        mode: 'no-cors'
      });

      if (this.config.debug) {
        console.log('[Vallor Tracker] Event sent:', eventName, 'ID:', eventId);
      }

      return eventId;
    } catch (e) {
      console.error('Vallor Tracker: Failed to send event', e);
      return null;
    }
  },

  // Monitorar cliques em WhatsApp (para negócios locais)
  monitorWhatsAppClicks: function() {
    document.addEventListener('click', async (e) => {
      const target = e.target.closest('a');
      if (!target) return;

      const href = target.getAttribute('href') || '';
      if (href.includes('wa.me') || href.includes('whatsapp.com') || href.includes('api.whatsapp.com')) {
        await this.sendEvent('Contact', {
          content_name: 'WhatsApp Button',
          contact_method: 'whatsapp',
          button_text: target.textContent.trim(),
          button_url: href
        });
      }
    });
  },

  // Monitorar envio de formulários
  monitorFormSubmissions: function() {
    document.addEventListener('submit', async (e) => {
      const form = e.target;
      if (!form.tagName || form.tagName !== 'FORM') return;

      // Capturar dados do formulário
      const formData = await this.captureFormData(form);

      await this.sendEvent('Lead', {
        content_name: form.id || form.name || 'Form Submission',
        form_id: form.id,
        form_name: form.name,
        ...formData
      });
    });
  },

  // Inicializar
  init: function(config = {}) {
    // Mesclar configuração
```

```
      this.config = { ...this.config, ...config };

      // Capturar script_key do data-attribute se não fornecido
      if (!this.config.scriptKey) {
        const scriptTag = document.currentScript || document.querySelector('script[data-vallor-key]');
        if (scriptTag) {
          this.config.scriptKey = scriptTag.getAttribute('data-vallor-key');
        }
      }

      // Definir URL da API se não fornecida
      if (!this.config.apiUrl) {
        const scriptTag = document.currentScript || document.querySelector('script[data-vallor-key]');
        if (scriptTag) {
          const scriptSrc = scriptTag.getAttribute('src');
          const url = new URL(scriptSrc, window.location.origin);
          this.config.apiUrl = `${url.origin}/api/track`;
        }
      }

      if (!this.config.scriptKey) {
        console.error('Vallor Tracker: script_key not found');
        return;
      }

      // Rastrear PageView automaticamente
      this.sendEvent('PageView');

      // Iniciar monitores
      this.monitorWhatsAppClicks();
      this.monitorFormSubmissions();

      if (this.config.debug) {
        console.log('[Vallor Tracker] Initialized with key:', this.config.scriptKey);
        console.log('[Vallor Tracker] API URL:', this.config.apiUrl);
      }
    }
  };

  // Expor globalmente
  window.VallorTracker = VallorTracker;

  // Auto-inicializar quando o DOM estiver pronto
  if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', function() {
      if (window.vallorConfig) {
        VallorTracker.init(window.vallorConfig);
      } else {
        VallorTracker.init();
      }
    });
  } else {
    if (window.vallorConfig) {
      VallorTracker.init(window.vallorConfig);
    } else {
      VallorTracker.init();
    }
  }
})();
```

5.2 Uso do Script no Site do Cliente

```
<!-- No <head> da landing page -->
<script
  src="https://tracker.seucliente.com/vallor-tracker.js"
  data-vallor-key="sk_live_abc123xyz"
  async>
</script>

<!-- Configuração opcional -->
<script>
  window.vallorConfig = {
    apiUrl: 'https://api.vallortracker.com/api/track',
    debug: false
  };
</script>

<!-- Eventos personalizados -->
<script>
  // Rastrear clique em botão
  document.getElementById('btn-comprar').addEventListener('click', function() {
    VallorTracker.sendEvent('InitiateCheckout', {
      content_name: 'Produto XYZ',
      value: 197.00,
```

```
      currency: 'BRL'
    });
  });

  // Rastrear visualização de produto
  VallorTracker.sendEvent('ViewContent', {
    content_name: 'Página de Vendas Produto XYZ',
    content_category: 'infoproduto'
  });
</script>
```

6. API Routes (Next.js)

6.1 Endpoint de Rastreamento

Arquivo: `app/api/track/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { adminSupabase } from '@/lib/supabase/admin';
import { sendToMetaCAPI } from '@/lib/meta/capi';

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();

    // Validar dados obrigatórios
    const { script_key, event_name, event_time, event_id } = body;

    if (!script_key || !event_name || !event_time) {
      return NextResponse.json(
        { error: 'Missing required fields' },
        { status: 400 }
      );
    }

    // Resolver tenant por script_key
    const { data: clientDomain, error: clientError } = await adminSupabase
      .from('client_domains')
      .select(`
        client_id,
        agency_id,
        clients!inner(
          id,
          name,
          meta_pixel_id,
          meta_access_token
        )
      `)
      .eq('script_key', script_key)
      .single();

    if (clientError || !clientDomain) {
      return NextResponse.json(
        { error: 'Invalid script_key' },
        { status: 404 }
      );
    }

    const clientId = clientDomain.client_id;
    const agencyId = clientDomain.agency_id;

    // Capturar IP real (considerar proxies)
    const ip = request.headers.get('x-forwarded-for')?.split(',')[0] ||
               request.headers.get('x-real-ip') ||
               'unknown';

    // Buscar ou criar visitante
    let visitorId = null;

    // 1. Tentar encontrar por fbp (mais confiável)
    if (body.fbp) {
      const { data: existingVisitor } = await adminSupabase
        .from('visitors')
        .select('id')
        .eq('client_id', clientId)
        .eq('fbp', body.fbp)
        .order('last_seen_at', { ascending: false })
        .limit(1)
        .single();
```

```
  if (existingVisitor) {
    visitorId = existingVisitor.id;

    // Atualizar last_seen_at
    await adminSupabase
      .from('visitors')
      .update({ last_seen_at: new Date().toISOString() })
      .eq('id', visitorId);
  }
}

// 2. Se não encontrou, criar novo visitante
if (!visitorId) {
  const { data: newVisitor } = await adminSupabase
    .from('visitors')
    .insert({
      agency_id: agencyId,
      client_id: clientId,
      fbp: body.fbp,
      fbc: body.fbc,
      ip: ip,
      user_agent: body.user_agent,
      fingerprint: body.fingerprint,
      fingerprint_hash: body.fingerprint_hash,
      landing_url: body.url,
      referrer: body.referrer,
      utm_source: body.utm_source,
      utm_medium: body.utm_medium,
      utm_campaign: body.utm_campaign,
      utm_content: body.utm_content,
      utm_term: body.utm_term
    })
    .select('id')
    .single();

  if (newVisitor) {
    visitorId = newVisitor.id;
  }
}

// Buscar campanha se UTM estiver presente
let campaignId = null;
if (body.utm_campaign) {
  const { data: campaign } = await adminSupabase
    .from('campaigns')
    .select('id')
    .eq('client_id', clientId)
    .or(`source.eq.${body.utm_source},name.ilike.%${body.utm_campaign}%`)
    .limit(1)
    .single();

  if (campaign) {
    campaignId = campaign.id;
  }
}

// Inserir evento
const { data: event, error: eventError } = await adminSupabase
  .from('tracking_events')
  .insert({
    agency_id: agencyId,
    client_id: clientId,
    visitor_id: visitorId,
    campaign_id: campaignId,
    event_name: event_name,
    event_time: new Date(event_time * 1000).toISOString(),
    event_id: event_id || `ev_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`,
    source_url: request.headers.get('referer'),
    page_url: body.url,
    referrer: body.referrer,
    ip: ip,
    user_agent: body.user_agent,
    fingerprint_hash: body.fingerprint_hash,
    external_id: body.external_id,
    click_id: body.fbclid || body.gclid,
    fbp: body.fbp,
    fbc: body.fbc,
    utm_source: body.utm_source,
    utm_medium: body.utm_medium,
    utm_campaign: body.utm_campaign,
    utm_content: body.utm_content,
    utm_term: body.utm_term,
    properties: body.properties || {}
  })
  .select()
  .single();
```

```
    if (eventError) {
      console.error('Error inserting event:', eventError);

      // Se for erro de duplicação, retornar sucesso
      if (eventError.code === '23505') { // unique_violation
        return NextResponse.json({
          success: true,
          event_id: event_id,
          message: 'Event already recorded (duplicate)'
        });
      }

      throw eventError;
    }

    // Enviar para Meta CAPI (async, não bloquear resposta)
    if (clientDomain.clients.meta_access_token && clientDomain.clients.meta_pixel_id) {
      sendToMetaCAPI(
        clientDomain.clients.meta_pixel_id,
        clientDomain.clients.meta_access_token,
        event
      ).catch(err => {
        console.error('Error sending to Meta CAPI:', err);
      });
    }

    return NextResponse.json({
      success: true,
      event_id: event.event_id,
      message: 'Event tracked successfully'
    });

  } catch (error) {
    console.error('Tracking error:', error);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}

// Opções de CORS
export async function OPTIONS(request: NextRequest) {
  return NextResponse.json({}, {
    headers: {
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Methods': 'POST, OPTIONS',
      'Access-Control-Allow-Headers': 'Content-Type'
    }
  });
}
```

6.2 Webhook Genérico de Compra

Arquivo: `app/api/webhook/purchase/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { adminSupabase } from '@/lib/supabase/admin';
import { attributeConversion } from '@/lib/attribution';
import { sendConversionToMetaCAPI } from '@/lib/meta/capi';

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const authHeader = request.headers.get('authorization');

    // Validar token de API
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const token = authHeader.replace('Bearer ', '');

    // Hash do token e buscar no banco
    const crypto = require('crypto');
    const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

    const { data: apiToken, error: tokenError } = await adminSupabase
      .from('api_tokens')
      .select('agency_id, client_id, scopes')
      .eq('token_hash', tokenHash)
      .is('revoked_at', null)
      .single();
```

```
if (tokenError || !apiToken) {
  return NextResponse.json({ error: 'Invalid token' }, { status: 401 });
}

// Verificar scope
if (!apiToken.scopes.includes('webhook:purchase')) {
  return NextResponse.json({ error: 'Insufficient permissions' }, { status: 403 });
}

// Validar dados obrigatórios
const { order_id, amount, currency = 'BRL', provider = 'generic' } = body;

if (!order_id || !amount) {
  return NextResponse.json(
    { error: 'Missing required fields: order_id, amount' },
    { status: 400 }
  );
}

// Verificar duplicação
const { data: existingConversion } = await adminSupabase
  .from('conversions')
  .select('id')
  .eq('client_id', apiToken.client_id)
  .eq('provider', provider)
  .eq('order_id', order_id)
  .single();

if (existingConversion) {
  return NextResponse.json({
    success: true,
    message: 'Conversion already recorded (duplicate)',
    conversion_id: existingConversion.id
  });
}

// Atribuir conversão a um visitante
const attribution = await attributeConversion(apiToken.client_id, {
  email: body.customer_email,
  phone: body.customer_phone,
  ip: body.customer_ip,
  fbp: body.customer_fbp,
  fbc: body.customer_fbc,
  fingerprint: body.customer_fingerprint
});

// Gerar event_id único
const eventId = `conv_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;

// Inserir conversão
const { data: conversion, error: conversionError } = await adminSupabase
  .from('conversions')
  .insert({
    agency_id: apiToken.agency_id,
    client_id: apiToken.client_id,
    visitor_id: attribution?.visitor_id || null,
    campaign_id: attribution?.campaign_id || null,
    adset_id: attribution?.adset_id || null,
    ad_id: attribution?.ad_id || null,
    provider: provider,
    order_id: order_id,
    status: body.status || 'approved',
    amount: amount,
    currency: currency,
    event_name: 'Purchase',
    event_id: eventId,
    external_id: body.external_id,
    click_id: body.customer_fbc,
    fbp: body.customer_fbp,
    fbc: body.customer_fbc,
    ip: body.customer_ip,
    user_agent: body.customer_user_agent,
    fingerprint_hash: body.customer_fingerprint,
    customer_email: body.customer_email,
    customer_phone: body.customer_phone,
    customer_first_name: body.customer_first_name,
    customer_last_name: body.customer_last_name,
    customer_city: body.customer_city,
    customer_state: body.customer_state,
    customer_zip: body.customer_zip,
    customer_country: body.customer_country || 'br',
    raw_payload: body,
    attributed_by: attribution?.method || 'unattributed',
    attributed_at: attribution ? new Date().toISOString() : null,
    conversion_time: body.conversion_time || new Date().toISOString()
  })
```

```
      .select()
      .single();

    if (conversionError) {
      console.error('Error inserting conversion:', conversionError);
      throw conversionError;
    }

    // Enviar para Meta CAPI
    sendConversionToMetaCAPI(apiToken.client_id, conversion)
      .catch(err => console.error('Error sending to Meta CAPI:', err));

    // Atualizar last_used_at do token
    await adminSupabase
      .from('api_tokens')
      .update({ last_used_at: new Date().toISOString() })
      .eq('token_hash', tokenHash);

    return NextResponse.json({
      success: true,
      conversion_id: conversion.id,
      attributed: !!attribution,
      attribution_method: attribution?.method || null
    });

  } catch (error) {
    console.error('Webhook error:', error);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

6.3 Webhook Kiwify

Arquivo: `app/api/webhook/kiwify/route.ts`

```
import { NextRequest, NextResponse } from 'next/server';
import { adminSupabase } from '@/lib/supabase/admin';

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const signature = request.headers.get('x-kiwify-signature');

    // Log do webhook
    await adminSupabase.from('webhook_logs').insert({
      provider: 'kiwify',
      event_type: body.event,
      external_event_id: body.order_id,
      status: 'received',
      request_headers: Object.fromEntries(request.headers.entries()),
      payload: body
    });

    // Mapear payload Kiwify para formato padrão
    const mappedData = {
      order_id: body.order_id,
      amount: body.amount || body.value,
      currency: body.currency || 'BRL',
      status: mapKiwifyStatus(body.status),
      customer_email: body.Customer?.email,
      customer_phone: body.Customer?.phone,
      customer_first_name: body.Customer?.first_name,
      customer_last_name: body.Customer?.last_name,
      customer_ip: body.Customer?.ip,
      customer_fbp: body.Customer?.fbp,
      customer_fbc: body.Customer?.fbc,
      product_name: body.Product?.name,
      product_id: body.Product?.id,
      provider: 'kiwify',
      conversion_time: body.created_at || new Date().toISOString()
    };

    // Encaminhar para webhook genérico
    const token = await getClientTokenByKiwifyConfig(body.Product?.id);

    const response = await fetch(`${process.env.NEXT_PUBLIC_APP_URL}/api/webhook/purchase`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
```

```
      },
      body: JSON.stringify(mappedData)
    });

    const result = await response.json();

    // Atualizar log
    await adminSupabase
      .from('webhook_logs')
      .update({
        status: result.success ? 'processed' : 'error',
        response_body: result
      })
      .eq('external_event_id', body.order_id)
      .eq('provider', 'kiwify');

    return NextResponse.json(result);

  } catch (error) {
    console.error('Kiwify webhook error:', error);

    await adminSupabase.from('webhook_logs').update({
      status: 'error',
      error_message: error.message
    }).eq('provider', 'kiwify');

    return NextResponse.json({ error: 'Processing failed' }, { status: 500 });
  }
}

function mapKiwifyStatus(status: string): string {
  const statusMap: Record<string, string> = {
    'paid': 'paid',
    'approved': 'approved',
    'refunded': 'refunded',
    'cancelled': 'cancelled',
    'chargeback': 'chargeback'
  };
  return statusMap[status?.toLowerCase()] || 'pending';
}

async function getClientTokenByKiwifyConfig(productId: string): Promise<string> {
  // Implementar lógica para buscar token baseado na configuração do cliente
  // Por exemplo, o cliente pode ter configurado o product_id na tabela de configurações
  const { data } = await adminSupabase
    .from('api_tokens')
    .select('token_hash')
    .limit(1)
    .single();

  return data?.token_hash || '';
}
```

7. Inteligência de Atribuição

7.1 Lógica de Match por Prioridade

> **Conceito:** O sistema tenta encontrar o visitante original que gerou a compra usando múltiplos critérios em ordem de confiabilidade.

Prioridade	Método	Confiabilidade	Descrição
1	fbc (Facebook Click ID)	Muito Alta (95%)	ID gerado quando usuário clica em anúncio do Facebook
2	fbp (Facebook Browser ID)	Alta (85%)	ID persistente do navegador no domínio
3	email_hash	Alta (80%)	Hash SHA256 do email do comprador
4	IP + User Agent	Média (60%)	Combinação de IP e navegador
5	fingerprint_hash	Média (55%)	Hash da digital do navegador
6	external_id	Baixa (40%)	ID do CRM/Lead se disponível

7.2 Arquivo: `lib/attribution.ts`

```
import { adminSupabase } from '@/lib/supabase/admin';
import crypto from 'crypto';

interface AttributionData {
  email?: string;
  phone?: string;
  ip?: string;
  fbp?: string;
  fbc?: string;
  fingerprint?: string;
  external_id?: string;
}

interface AttributionResult {
  visitor_id: string;
  campaign_id?: string;
  adset_id?: string;
  ad_id?: string;
  method: string; // fbc, fbp, email, ip, fingerprint, external_id
}

/**
 * Atribui uma conversão a um visitante usando múltiplos métodos
 * @param clientId - ID do cliente
 * @param data - Dados para match
 * @returns Resultado da atribuição ou null se não encontrou
 */
export async function attributeConversion(
  clientId: string,
  data: AttributionData
): Promise<AttributionResult | null> {

  // Janela de atribuição: 30 dias
  const attributionWindow = new Date();
  attributionWindow.setDate(attributionWindow.getDate() - 30);

  // 1. Tentar match por fbc (Facebook Click ID) - PRIORIDADE MÁXIMA
  if (data.fbc) {
    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('fbc', data.fbc)
      .gte('last_seen_at', attributionWindow.toISOString())
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'fbc'
      };
    }
  }

  // 2. Tentar match por fbp (Facebook Browser ID)
  if (data.fbp) {
    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('fbp', data.fbp)
      .gte('last_seen_at', attributionWindow.toISOString())
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'fbp'
      };
    }
  }

  // 3. Tentar match por email_hash
```

```
  if (data.email) {
    const emailHash = crypto.createHash('sha256')
      .update(data.email.toLowerCase().trim())
      .digest('hex');

    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('email_hash', emailHash)
      .gte('last_seen_at', attributionWindow.toISOString())
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'email'
      };
    }
  }

  // 4. Tentar match por IP + User Agent (menos confiável)
  if (data.ip) {
    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('ip', data.ip)
      .gte('last_seen_at', attributionWindow.toISOString())
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'ip'
      };
    }
  }

  // 5. Tentar match por fingerprint
  if (data.fingerprint) {
    const fingerprintHash = crypto.createHash('sha256')
      .update(data.fingerprint)
      .digest('hex');

    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('fingerprint_hash', fingerprintHash)
      .gte('last_seen_at', attributionWindow.toISOString())
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'fingerprint'
      };
    }
  }

  // 6. Tentar match por external_id (se disponível)
  if (data.external_id) {
    const { data: visitor } = await adminSupabase
      .from('visitors')
      .select('id, campaign_id, utm_campaign')
      .eq('client_id', clientId)
      .eq('external_id', data.external_id)
      .gte('last_seen_at', attributionWindow.toISOString())
```

```
      .order('last_seen_at', { ascending: false })
      .limit(1)
      .single();

    if (visitor) {
      const campaignData = await getCampaignData(clientId, visitor.utm_campaign);
      return {
        visitor_id: visitor.id,
        campaign_id: campaignData?.campaign_id,
        adset_id: campaignData?.adset_id,
        ad_id: campaignData?.ad_id,
        method: 'external_id'
      };
    }
  }

  // Não encontrou correspondência
  return null;
}

/**
 * Busca dados da campanha por nome ou ID
 */
async function getCampaignData(clientId: string, utmCampaign?: string) {
  if (!utmCampaign) return null;

  const { data: campaign } = await adminSupabase
    .from('campaigns')
    .select(`
      id,
      adsets(id, ads(id))
    `)
    .eq('client_id', clientId)
    .or(`name.ilike.%${utmCampaign}%,source.eq.${utmCampaign}`)
    .limit(1)
    .single();

  if (!campaign) return null;

  return {
    campaign_id: campaign.id,
    adset_id: campaign.adsets?.[0]?.id,
    ad_id: campaign.adsets?.[0]?.ads?.[0]?.id
  };
}
```

8. Dashboard Analítico

8.1 Cards de Métricas

Arquivo: `components/dashboard/metric-cards.tsx`

```
'use client';

import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { DollarSign, ShoppingCart, Users, TrendingUp } from 'lucide-react';

interface MetricCardsProps {
  data: {
    investment: number;
    conversions: number;
    revenue: number;
    cpa: number;
    roas: number;
  };
}

export function MetricCards({ data }: MetricCardsProps) {
  return (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
      <Card>
        <CardHeader className="flex flex-row items-center justify-between pb-2">
          <CardTitle className="text-sm font-medium">Investimento</CardTitle>
          <DollarSign className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">
            {new Intl.NumberFormat('pt-BR', {
              style: 'currency',
```

```
              currency: 'BRL'
            }).format(data.investment)}
          </div>
        </CardContent>
      </Card>

      <Card>
        <CardHeader className="flex flex-row items-center justify-between pb-2">
          <CardTitle className="text-sm font-medium">Conversões</CardTitle>
          <ShoppingCart className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">{data.conversions}</div>
          <p className="text-xs text-muted-foreground mt-1">
            Receita: {new Intl.NumberFormat('pt-BR', {
              style: 'currency',
              currency: 'BRL'
            }).format(data.revenue)}
          </p>
        </CardContent>
      </Card>

      <Card>
        <CardHeader className="flex flex-row items-center justify-between pb-2">
          <CardTitle className="text-sm font-medium">CPA Médio</CardTitle>
          <Users className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">
            {new Intl.NumberFormat('pt-BR', {
              style: 'currency',
              currency: 'BRL'
            }).format(data.cpa)}
          </div>
        </CardContent>
      </Card>

      <Card>
        <CardHeader className="flex flex-row items-center justify-between pb-2">
          <CardTitle className="text-sm font-medium">ROAS</CardTitle>
          <TrendingUp className="h-4 w-4 text-muted-foreground" />
        </CardHeader>
        <CardContent>
          <div className="text-2xl font-bold">
            {data.roas.toFixed(2)}x
          </div>
          <p className="text-xs text-muted-foreground mt-1">
            {data.roas > 3 ? '🟢 Excelente' : data.roas > 2 ? '🟡 Bom' : '🔴 Atenção'}
          </p>
        </CardContent>
      </Card>
    </div>
  );
}
```

8.2 Gráfico de Funil

Arquivo: `components/dashboard/funnel-chart.tsx`

```
'use client';

import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Cell } from 'recharts';

interface FunnelData {
  name: string;
  value: number;
  percentage: number;
}

interface FunnelChartProps {
  data: FunnelData[];
}

const COLORS = ['#3b82f6', '#8b5cf6', '#ec4899', '#10b981'];

export function FunnelChart({ data }: FunnelChartProps) {
  return (
    <Card>
      <CardHeader>
        <CardTitle>Funil de Conversão</CardTitle>
      </CardHeader>
      <CardContent>
```

```
        <div className="h-[300px] w-full">
          <ResponsiveContainer width="100%" height="100%">
            <BarChart data={data} layout="vertical">
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis type="number" />
              <YAxis dataKey="name" type="category" width={120} />
              <Tooltip
                formatter={(value: number, name: string, props: any) => [
                  `${value.toLocaleString('pt-BR')} (${props.payload.percentage.toFixed(1)}%)`,
                  'Eventos'
                ]}
              />
              <Bar dataKey="value" radius={[0, 4, 4, 0]}>
                {data.map((entry, index) => (
                  <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
                ))}
              </Bar>
            </BarChart>
          </ResponsiveContainer>
        </div>
      </CardContent>
    </Card>
  );
}
```

Próximas Seções

As seções seguintes (Segurança, Escalabilidade, Deploy) continuam com o mesmo nível de detalhamento técnico.

Documento Completo: Este é um documento extenso e profissional de arquitetura de software. Para visualizar o documento completo com todas as seções, acesse:

- Documento Completo no GenSpark

Vallor Tracker - Documento de Arquitetura de Software v10.0

Sistema Multi-Tenant de Rastreamento e Atribuição | Janeiro 2026

Confidencial - Uso Interno

Feito com Genspark